

Documentação do Compilador

Projeto de Compiladores — Versão 2026

Disciplina: Compiladores

Professor: Marcus Ramos

Alunos: Hellian S.S Peixinho

Nadjon Filipe Santos Oliveira

Matheus Vinicius Rodrigues Valadares Barros

Juazeiro/Petrolina — BA, 2026

Sumário

1	Linguagem-Fonte	2
1.1	Relação e Estrutura de Separadores	3
1.2	Sintaxe Livre de Contexto	3
1.3	Sintaxe Dependente de Contexto	3
1.4	Semântica	4
2	Descrição Geral da Arquitetura do Compilador	4
2.1	Analizador Léxico (Scanner)	4
2.2	Analizador Sintático (Parser)	5
2.3	Printagem da Árvore (Printer)	5
2.4	Analizador Semântico (Checker)	5
2.5	Gerador de Código (CodeGenerator)	5
3	Fundamentação Teórica e Técnicas Empregadas na Análise Sintática	5
3.1	Gramática Livre de Contexto	5
3.2	Análise Descendente e Recursão	6
3.3	Conjunto de Primeiro e Símbolos de Sincronização	6
3.4	Manipulação de Erros	6
3.5	Resultado: Árvore de Análise Sintática	6
4	Análise Léxica	6
4.1	Relação de Tokens	6
4.2	Gramática Léxica	7
4.3	Manipulações Efetuadas na Gramática Original e Justificativas	7
4.4	Verificação da Condição LL(1) na Gramática Manipulada	7
4.5	Técnicas Utilizadas e Visão Geral do Funcionamento	8
4.6	Exemplos de Entradas e Saídas	8
5	Análise Sintática	8
5.1	Gramática Sintática	8
5.2	Manipulações Efetuadas na Gramática Original e Justificativas	8
5.3	Verificação da Condição LL(1) na Gramática Manipulada	9
5.4	Técnicas Utilizadas e Visão Geral do Funcionamento	9
5.5	Exemplos de Entradas e Saídas	9
6	Descrição das Principais Estruturas de Dados Utilizadas	10
6.1	Tabela de Símbolos	10
6.2	Árvore Sintática Abstrata (AST)	10
6.3	Representação de Atributos	10
6.4	Representação de Tokens	11
7	Montagem e Impressão da Árvore de Sintaxe Abstrata	11
7.1	Estruturas de Dados e Algoritmos Utilizados	11
7.2	Exemplo de Programa-Fonte e Árvore Gerada	11

8	Análise de Contexto	12
8.1	Identificação	12
8.2	Verificação de Tipos	12
8.3	Exemplos de Entradas e Saídas	13
9	Ambiente de Execução	13
9.1	Tipo de Máquina Usada	13
9.2	Avaliação de Expressões	13
9.3	Representação de Dados	14
9.4	Alocação de Memória	14
9.5	Passagem de Parâmetros e Retorno de Valor de Função	14
10	Linguagem-Objeto	14
11	Geração de Código	15
11.1	Funções de Código Implementadas	15
11.2	Descrição das Estruturas de Dados e Algoritmos Utilizados	15
11.3	Alocação de Memória para a Máquina-Objeto	15
11.4	Exemplos de Tradução com Trechos Representativos	15
12	Manual de Instalação	16
13	Manual de Compilação	16
14	Manual de Utilização	17
15	Conclusões Finais	19

1 Linguagem-Fonte

Gramática sintática da linguagem (versão 2026, com extensões implementadas):

```

<programa> ::= program <id> ; <corpo> .
<corpo> ::= <declarações> <comando-composto>
<declarações> ::= (<declaração> ;)*
<declaração> ::= <declaração-de-variável>
<declaração-de-variável> ::= var <lista-de-ids> : <tipo>
<lista-de-ids> ::= <id> (, <id>)*
<tipo> ::= <tipo-simples>
<comando> ::= <atribuição> | <condicional> | <iterativo> | <comando-composto>
<atribuição> ::= <variável> := <expressão>
<comando-composto> ::= begin <lista-de-comandos> end
<lista-de-comandos> ::= (<comando> ;)*
<condicional> ::= if <expressão> then <comando> ( else <comando> | <vazio> )
<iterativo> ::= while <expressão> do <comando>
<expressão> ::= <expressão-simples> (<op-rel> <expressão-simples> | <vazio>)
<expressão-simples> ::= (<termo>)(<op-ad><termo>)*
<termo> ::= (<fator>)(<op-mul><fator>)*
<fator> ::= <variável> | <literal> | "(" <expressão> ")"
<variável> ::= <id>

```

1.1 Relação e Estrutura de Separadores

A linguagem utiliza espaços em branco, tabulações e quebras de linha como separadores léxicos, usados para delimitar palavras-chave, identificadores, operadores e demais símbolos. Comentários iniciam com o caractere “!” e se estendem até o fim da linha; são tratados como separadores e integralmente ignorados pelo compilador.

```
<Separator> ::= <espaço-vazio> | \t | \n | <Comment>
<Comment>   ::= ! <CommentLine> EOL
<CommentLine> ::= Graphic <CommentLine> | Graphic
```

1.2 Sintaxe Livre de Contexto

A sintaxe da linguagem é livre de contexto: as regras sintáticas são definidas independentemente do local onde as construções aparecem no código-fonte. Essas regras são especificadas por meio da Gramática Livre de Contexto (GLC) apresentada no início desta seção, que define como os diferentes elementos da linguagem podem ser combinados para formar programas válidos.

1.3 Sintaxe Dependente de Contexto

Apesar de a sintaxe ser livre de contexto, a linguagem possui regras dependentes de contexto, verificadas na fase de análise semântica (Seção 8):

Verificação de declaração de variáveis

- Variáveis devem ser declaradas com um tipo válido, que pode ser `boolean`, `integer` ou `real`;
- Variáveis devem ser declaradas no início do bloco (antes do comando composto principal);
- Variáveis só podem ser declaradas uma única vez; identificadores não podem se repetir, mesmo com tipos diferentes.

Verificação de tipos das expressões

- Os tipos dos operandos devem ser compatíveis com os operadores utilizados;
- Em atribuições, o tipo da expressão deve ser compatível com o tipo da variável que a recebe;
- Operações relacionais sempre resultam no tipo `boolean`;
- Em operações não relacionais entre `integer` e `real`, o tipo resultante é sempre `real`;
- Operações entre operandos de mesmo tipo resultam nesse mesmo tipo (exceto relacionais).

Verificação de escopo

- A linguagem possui escopo único e monolítico para todo o programa;

- Todas as declarações são visíveis nesse único escopo, do início ao fim da execução;
- As variáveis permanecem alocadas durante todo o ciclo de vida do programa.

Verificação de expressões condicionais e iterativas

- Toda expressão avaliada em um comando condicional ou iterativo deve obrigatoriamente ser do tipo `boolean`.

Nota sobre escopo

Esta é a definição de escopo efetivamente implementada pelo `Checker/IdentificationTable` (tabela de símbolos única, sem pilha de escopos).

1.4 Semântica

O significado de cada construção da linguagem é definido a seguir:

Construção	Semântica
Atribuição	Avalia a expressão à direita e armazena o valor resultante na variável indicada.
Comando composto	Executa sequencialmente todos os comandos da lista, na ordem em que aparecem.
Condicional	Executa o comando após “then” se a expressão for verdadeira; se houver “else” e a expressão for falsa, executa o comando correspondente.
Iterativo	Executa repetidamente o comando associado enquanto a expressão permanecer verdadeira.
Expressão	Avalia a expressão simples; se houver operador relacional e uma segunda expressão simples, compara ambas e retorna um <code>boolean</code> .
Expressão simples / Termo	Avalia termos/fatores da esquerda para a direita, aplicando os operadores aditivos/multiplicativos correspondentes.
Fator	Unidade básica de uma expressão: uma variável, um literal, ou uma expressão entre parênteses.
Programa	Unidade principal, iniciada por “program <id>;” seguida do corpo (declarações + comando composto) e finalizada por “.”

2 Descrição Geral da Arquitetura do Compilador

Um compilador traduz o código-fonte de uma linguagem de alto nível em código-objeto executável em uma máquina-alvo. A arquitetura deste compilador segue o modelo clássico em cinco fases sequenciais, cada uma implementada em um pacote Java dedicado, orquestradas pela classe principal `Compiler`:

2.1 Analisador Léxico (Scanner)

Lê o código-fonte caractere por caractere e o converte em uma sequência de tokens — unidades léxicas como palavras-chave, identificadores, literais e operadores — descartando

espaços em branco e comentários. Erros léxicos (caracteres ou sequências inválidas) resultam no token `Token.ERROR`, reportado ao final da varredura com a linha e coluna correspondentes.

2.2 Analisador Sintático (Parser)

Recebe a sequência de tokens do `Scanner` e constrói a Árvore Sintática Abstrata (AST), verificando se o programa segue a gramática da linguagem. Implementado por descida recursiva, com um método por não-terminal da gramática LL(1). Erros sintáticos são reportados com a posição exata do símbolo não aceito.

2.3 Printagem da Árvore (Printer)

Percorre a AST já construída e imprime sua estrutura hierárquica de forma legível no terminal, usando indentação para representar a profundidade de cada nó. Foi o primeiro ponto do compilador em que o padrão de projeto *Visitor* foi aplicado.

2.4 Analisador Semântico (Checker)

Verifica se o programa está de acordo com as regras de escopo e de tipos da linguagem (Seção 1.3), construindo uma tabela de símbolos durante a identificação de declarações e verificando a compatibilidade de tipos em atribuições e expressões. Também utiliza o padrão *Visitor*.

2.5 Gerador de Código (CodeGenerator)

Traduz a AST, já validada semanticamente, para instruções da máquina-objeto TAM (*Triangle Abstract Machine*), gravando o resultado em um arquivo de texto (`Object-code.txt`). Também percorre a AST através do padrão *Visitor*.

3 Fundamentação Teórica e Técnicas Empregadas na Análise Sintática

A análise sintática verifica se o código-fonte segue a estrutura gramatical da linguagem, produzindo uma representação intermediária (a AST) utilizada pelas etapas seguintes de compilação. Este compilador utiliza a técnica de análise sintática descendente recursiva.

3.1 Gramática Livre de Contexto

A Gramática Livre de Contexto (GLC) da linguagem, apresentada na Seção 1, foi traduzida diretamente em métodos Java: cada não-terminal corresponde a um método (por exemplo, `parse_programa`, `parse_comando`, `parse_expressao`), responsável por reconhecer e processar a respectiva construção sintática.

3.2 Análise Descendente e Recursão

A análise sintática descendente parte do símbolo inicial da gramática (<programa>) e tenta derivar a cadeia de entrada, avançando pela árvore de cima para baixo. A recursão entre os métodos do `Parser` permite reconhecer construções aninhadas, como expressões dentro de expressões e comandos dentro de comandos compostos.

3.3 Conjunto de Primeiro e Símbolos de Sincronização

Os conjuntos *first1()* (detalhados na Seção 5.3) indicam quais tokens podem iniciar cada construção sintática, permitindo ao `Parser` decidir qual regra aplicar apenas observando o token atual (análise preditiva). Os métodos `accept` e `acceptIt` funcionam como pontos de sincronização, consumindo o token esperado e avançando para o próximo.

3.4 Manipulação de Erros

Quando um token inesperado é encontrado, o `Parser` emite uma mensagem de erro contendo o símbolo não aceito e sua posição exata (linha e coluna) no código-fonte, por meio da classe `ShowError`, interrompendo a compilação.

3.5 Resultado: Árvore de Análise Sintática

O produto final da análise sintática é a AST, uma representação hierárquica do programa-fonte que serve de base para as etapas seguintes: impressão da árvore, análise de contexto e geração de código.

4 Análise Léxica

Implementada na classe `Scanner`, converte o código-fonte em uma sequência de tokens.

4.1 Relação de Tokens

Constante	Valor	Significado
IDENTIFIER	0	Identificador
BOOLLITERAL	1	Literal booleano
INTLITERAL	2	Literal inteiro
FLOATLITERAL	3	Literal de ponto flutuante
BEGIN ... END	4–12	Palavras-chave: begin, if, then, else, var, while, do, program, end
BECOMES	13	:=
LPAREN / RPAREN	14 / 15	(e)
COLON / SEMICOLON / COMMA / PERIOD	16–19	: ; , .
EXCLAMATION / ARROBA / HASHTAG / ELLIPSIS	20–23	! @ # ...
RELATIONALOPERATOR	24	<, >, <=, >=, =, <>
ADDITIONALOPERATOR	25	+, -, or

Constante	Valor	Significado
MULTIPLICATIONALOPERATOR	26	*, /, and
TIPOSIMPLES	27	integer, real, boolean
EOT / ERROR	28 / 29	Fim de texto / token inválido

4.2 Gramática Léxica

```

<id>      ::= <letra> (<letra> | <digito>)*
<letra>   ::= a | b | ... | z
<digito>  ::= 0 | 1 | ... | 9
<int-lit> ::= <digito>(<digito>)*
<float-lit> ::= <int-lit> . <int-lit> | <int-lit> . | . <int-lit>
<bool-lit> ::= true | false
<literal> ::= <bool-lit> | <int-lit> | <float-lit>
<tipo-simples> ::= integer | real | boolean
<op-ad>   ::= + | - | or
<op-mul>  ::= * | / | and
<op-rel>  ::= < | > | <= | >= | = | <>
<outros>  ::= ! | @ | # | ...

```

4.3 Manipulações Efetuadas na Gramática Original e Justificativas

A gramática lexical fornecida pelo professor para 2026 definia apenas os tipos `integer/boolean`, o operador relacional restrito a `<`, `>` e `=`, e a produção `<float-lit>` sem uso efetivo (não conectada a `<literal>`). Foram feitas as seguintes extensões:

- **Inclusão do tipo real:** necessário para representar valores não inteiros (ex.: conversões de temperatura, médias), amplamente úteis nos casos de teste da disciplina.
- **Operadores relacionais compostos (`<=`, `>=`, `<>`):** completam o conjunto usual de comparações e simplificam a escrita de condições, sem introduzir ambiguidade na gramática LL(1).
- **Conexão de `<float-lit>` em `<literal>`:** sem essa ligação a produção ficaria órfã e nenhum literal de ponto flutuante poderia de fato ser reconhecido.

Essas extensões preservam a compatibilidade com todos os programas válidos na gramática mínima fornecida — todo programa aceito pela gramática original também é aceito por esta versão estendida.

4.4 Verificação da Condição LL(1) na Gramática Manipulada

A análise léxica não exige verificação de condição LL(1): diferentemente da análise sintática, ela não lida com ambiguidade ou recursividade entre regras, apenas com o reconhecimento direto de padrões de caracteres em cada categoria de token.

4.5 Técnicas Utilizadas e Visão Geral do Funcionamento

A varredura é feita pelo método `scan()` da classe `Scanner`, que primeiro ignora separadores (espaços, tabulações, quebras de linha e comentários) e então despacha para uma função de reconhecimento específica conforme o primeiro caractere não separador: `scanIdentifierOrReservedWord()`, `scanNumber()` ou `scanRelationalOperator()`, entre outras. Cada token retornado carrega seu tipo, lexema, linha e coluna.

4.6 Exemplos de Entradas e Saídas

Exemplo 1 — Cadeia Bem-Formada

```
program example;
var x: integer;
begin
  x := 42;
end
.
```

Tokens gerados: PROGRAM, IDENTIFIER(example), SEMICOLON, VAR, IDENTIFIER(x), COLON, TIPOSIMPLES, SEMICOLON, BEGIN, IDENTIFIER(x), BECOMES, INTLITERAL(42), SEMICOLON, END, PERIOD.

Exemplo 2 — Cadeia Mal-Formada

```
program invalido;
var y := 10;
begin
  y := 20;
end
.
```

O uso de “:=” logo após “var y” (posição inválida para esse contexto) gera um token `Token.ERROR` durante a varredura.

5 Análise Sintática

5.1 Gramática Sintática

A gramática sintática utilizada pelo Parser é a apresentada na Seção 1 deste documento (já na forma LL(1), com `lista-de-ids` conectada à declaração de variável).

5.2 Manipulações Efetuadas na Gramática Original e Justificativas

A gramática original (fornecida antes da transformação LL(1)) apresentava recursão à esquerda em `<expressão-simples>`, `<termo>` e `<declarações>`, além de conjuntos *first()* sobrepostos em diversas produções. As seguintes manipulações foram aplicadas:

- **Eliminação de recursão à esquerda:** $\langle \text{expressão-simples} \rangle ::= \langle \text{termo} \rangle (\langle \text{op-ad} \rangle \langle \text{termo} \rangle)^*$ e $\langle \text{termo} \rangle ::= \langle \text{fator} \rangle (\langle \text{op-mul} \rangle \langle \text{fator} \rangle)^*$, reescritas em forma iterativa (EBNF) equivalente.
- **Fatoração à esquerda:** aplicada onde múltiplas alternativas compartilhavam prefixo comum, garantindo decisão determinística com um único token de *lookahead*.
- **Introdução de produção vazia ($\langle \text{vazio} \rangle$):** usada em $\langle \text{condicional} \rangle$ (else opcional) e em $\langle \text{expressão} \rangle$ (segunda parte relacional opcional).
- **Conexão de $\langle \text{lista-de-ids} \rangle$:** ligada a $\langle \text{declaração-de-variável} \rangle$, permitindo declarar múltiplas variáveis do mesmo tipo em uma única linha (ex.: `var v1, v2, v3 : integer;`), como implementado em `Parser.parse_declaracaoDeVariavel`.

5.3 Verificação da Condição LL(1) na Gramática Manipulada

A condição LL(1) exige que, para cada não-terminal, os conjuntos $first()$ de suas alternativas sejam disjuntos, permitindo decidir a produção a aplicar observando apenas o próximo token. A tabela a seguir apresenta os conjuntos $first1()$ da gramática manipulada:

Não-terminal	First1()
$\langle \text{programa} \rangle$	{program}
$\langle \text{corpo} \rangle$	{var, begin}
$\langle \text{declarações} \rangle$	{var, begin}
$\langle \text{declaração-de-variável} \rangle$	{var}
$\langle \text{comando} \rangle$	{ $\langle \text{id} \rangle$ } $\cup$ {if} $\cup$ {while} $\cup$ {begin}
$\langle \text{comando-composto} \rangle$	{begin}
$\langle \text{condicional} \rangle$	{if}
$\langle \text{iterativo} \rangle$	{while}
$\langle \text{expressão} \rangle$	{ $\langle \text{id} \rangle$, $\langle \text{literal} \rangle$, "("}
$\langle \text{expressão-simples} \rangle$	{ $\langle \text{id} \rangle$, $\langle \text{literal} \rangle$, "("}
$\langle \text{termo} \rangle$	{ $\langle \text{id} \rangle$, $\langle \text{literal} \rangle$, "("}
$\langle \text{fator} \rangle$	{ $\langle \text{id} \rangle$ } $\cup$ { $\langle \text{literal} \rangle$ } $\cup$ {"("}
$\langle \text{tipo-simples} \rangle$	{integer, real, boolean}
$\langle \text{op-rel} \rangle$	{<, >, <=, >=, =, <>}

5.4 Técnicas Utilizadas e Visão Geral do Funcionamento

Cada não-terminal é mapeado para um método na classe `Parser` (`parse_programa`, `parse_corpo`, `parse_declaracoes`, `parse_comando`, `parse_expressao`, `parse_termo`, `parse_fator`, etc.), que consome tokens via `accept()`/`acceptIt()` e retorna diretamente o nó correspondente da AST, já construída durante o próprio parsing.

5.5 Exemplos de Entradas e Saídas

Cadeia bem formada (aceitação)

```

program myProgram;
var x: integer;
begin
  x := 5;
end
.

```

Cadeia mal formada (rejeição)

```

program myProgram;
begin
  x := 5;
.

```

Neste exemplo, o “end” do comando composto está ausente antes do ponto final; o Parser detecta o símbolo inesperado e reporta a posição exata do erro.

6 Descrição das Principais Estruturas de Dados Utilizadas

6.1 Tabela de Símbolos

A classe `IdentificationTable` armazena, em uma lista de objetos `Attribute` (identificador, tipo e nó de declaração), todas as variáveis declaradas no programa. Os métodos principais são:

- `enter(id, tipo, declaracao)` — insere um novo identificador na tabela;
- `retrieve(id)` — busca um identificador já declarado, retornando `null` caso não exista.

Como a linguagem possui escopo único (Seção 1.3), a tabela é global durante toda a análise de contexto — não há pilha de escopos nem remoção de identificadores ao final de blocos.

6.2 Árvore Sintática Abstrata (AST)

Representação hierárquica do programa, construída por cerca de 25 classes de nó (pacote `visitor`), como `nodePrograma`, `nodeComando` e suas subclasses, `nodeExpressao`, `nodeTermo` e `nodeFator`. Cada classe implementa o método `visit(Visitor)`, permitindo que diferentes fases (impressão, checagem, geração de código) percorram a mesma árvore sem alterar sua estrutura — aplicação direta do padrão de projeto *Visitor*.

6.3 Representação de Atributos

A classe `Attribute` associa a cada identificador seu tipo (`Type`) e o nó de declaração correspondente (`nodeDeclaracaoDeVariavel`), permitindo que a informação de tipo seja recuperada durante a verificação semântica sempre que a variável é referenciada.

6.4 Representação de Tokens

A classe `Token` encapsula o tipo do token (`kind`), o lexema textual (`spelling`) e a posição no código-fonte (`line`, `column`), sendo o principal meio de comunicação entre o `Scanner` e o `Parser`.

7 Montagem e Impressão da Árvore de Sintaxe Abstrata

7.1 Estruturas de Dados e Algoritmos Utilizados

Classe `Printer`: implementa a interface `Visitor` e é responsável por imprimir a AST no console.

Indentação: controlada pelas variáveis `indentacao` (nível atual) e `tabulacao`, usando a string “| ” repetida para representar a profundidade de cada nó.

Algoritmo de impressão: visita recursiva — cada nó, ao ser visitado, imprime seu próprio rótulo e então delega a visita aos seus filhos, que imprimem com um nível a mais de indentação.

7.2 Exemplo de Programa-Fonte e Árvore Gerada

Exemplo executado e verificado diretamente no compilador (versão 2026, com múltiplos identificadores por declaração já implementado):

```
program codigo;  
var variavel : integer;  
  
begin  
    variavel := 10 + 5;  
end  
.
```

Saída real da impressão da AST:

```
---> Impressao da arvore  
Programa:  
ID: codigo  
| | Corpo:  
| | Declaracoes:  
| | | | | Declaracoes:  
| | | | | Declaracao 1:  
| | | | | Declaracao:  
| | | | | Declaracao de Variavel:  
| | | | | | IDs:  
| | | | | | | | variavel  
| | | | | | | Tipo:  
| | | | | | | integer  
| | Comando composto:  
| | | variavel  
| | | Expressao Simples 1:
```

```

| | | | Termo:
| | | | | 10
| | | | Operador Aditivo:
| | | | | +
| | | | Termo:
| | | | | 5

```

Nota

Note o nível “IDs:” na declaração — presente porque a declaração de variável aceita uma lista de identificadores (ver Seção 5.2).

8 Análise de Contexto

Verifica se o programa segue as regras de escopo e de tipos da linguagem (Seção 1.3), dividida em duas etapas: identificação e verificação de tipos, ambas implementadas na classe `Checker` através do padrão *Visitor*.

8.1 Identificação

Durante a visita aos nós de declaração, o `Checker` registra cada identificador na `IdentificationTable`, associando-o a seu tipo. Uma tentativa de declarar o mesmo identificador mais de uma vez é barrada com erro.

Exemplo (erro real, gerado pelo compilador):

```

program codigo;
var x : integer;
var x : real;

begin
  x := 1;
end
.

```

```

ERRO: Variavel "x" ja declarada
Linha: 3 Coluna: 5

```

8.2 Verificação de Tipos

Ao visitar comandos de atribuição, condicionais e iterativos, o `Checker` calcula o tipo de cada expressão (`Type.evaluate`) e verifica sua compatibilidade com o contexto: tipo da variável, no caso de atribuições; tipo `boolean`, no caso de condições.

Exemplo (erro real, gerado pelo compilador):

```

program codigo;
var x : integer;

begin

```

```
x := true;
end
.
```

ERRO: Tipo de atribuicao invalida: x...
Linha: 5 Coluna: 3

8.3 Exemplos de Entradas e Saídas

Cadeia Bem Formada

```
program Example;
var x: integer;
begin
  x := 5;
  if x > 0 then
    x := x - 1;
end
.
```

Cadeia Mal Formada

```
program Example;
var x: integer;
begin
  x := true;
end
.
```

Neste exemplo, o programa é sintaticamente válido, mas semanticamente incorreto: atribui um valor boolean a uma variável do tipo `integer`.

9 Ambiente de Execução

9.1 Tipo de Máquina Usada

O código gerado é destinado à TAM (*Triangle Abstract Machine*), a máquina abstrata de pilha definida no livro-texto de referência (*Programming Language Processors in Java*).

9.2 Avaliação de Expressões

As expressões são avaliadas por meio de instruções de pilha: operandos são empilhados (`LOAD/LOADL`) e operadores consomem os dois valores do topo, empilhando o resultado (`CALL ADD, CALL MULT`, etc.), na ordem determinada pela precedência da gramática (relacional < aditivo < multiplicativo).

9.3 Representação de Dados

Cada variável é mapeada para um endereço relativo (deslocamento) na pilha, armazenado nos mapas `variavelEnderecoMap` (nome $\rightarrow$ endereço) e `variavelTipoMap` (nome $\rightarrow$ tipo), populados durante a geração de código das declarações.

9.4 Alocação de Memória

Tipo	Tamanho alocado
boolean	1 unidade
integer	2 unidades
real	2 unidades

A alocação ocorre via instrução `PUSH` no início do programa (uma por variável declarada) e é desfeita via `POP` ao final da execução, pelo método `freeMemory()`.

9.5 Passagem de Parâmetros e Retorno de Valor de Função

A linguagem-fonte não define procedimentos nem funções, portanto não há passagem de parâmetros nem retorno de valores a implementar.

10 Linguagem-Objeto

A linguagem-objeto utilizada é a TAM, especificada no livro *Programming Language Processors in Java*. As instruções empregadas por este compilador são:

Instrução	Efeito
LOAD (n) d[r]	Carrega um objeto de n palavras do endereço ($d +$ registrador r) e empilha.
LOADL d	Empilha o valor literal d (1 palavra).
STORE (n) d[r]	Desempilha um objeto de n palavras e armazena no endereço ($d +$ registrador r).
CALL (n) d[r]	Chama a rotina no endereço indicado (usado para operadores como ADD, SUB, GT, etc.).
PUSH d	Empilha d palavras não inicializadas (aloca espaço para variáveis).
POP (n) d	Desempilha o resultado de n palavras, descarta d palavras adicionais e reempilha o resultado.
JUMP d[r]	Desvia incondicionalmente para o endereço de código indicado.
JUMPIF (n) d[r]	Desempilha um valor; se igual a n , desvia para o endereço indicado.
HALT	Encerra a execução do programa.

11 Geração de Código

11.1 Funções de Código Implementadas

Implementadas na classe `CodeGenerator`, através do padrão *Visitor*:

- `visit_nodeComandoAtribuicao` — traduz atribuição em instrução de armazenamento (STORE).
- `visit_nodeComandoCondicional` — traduz comando condicional em instruções de desvio (JUMPIF/JUMP) com rótulos.
- `visit_nodeComandoIterativo` — traduz comando de repetição em instruções de loop com rótulos.
- `visit_nodeDeclaracaoDeVariavel` — traduz declaração de variável em instrução de alocação (PUSH).
- `visit_nodeExpressao` / `visit_nodeOperador` / `visit_nodeLiteral` — traduzem expressões, operadores e literais em instruções intermediárias (LOADL, CALL).
- `openFile` / `closeFile` / `writeCodeLine` — controlam a escrita do arquivo de saída `Object-code.txt`.
- `freeMemory` — gera instruções POP para desalocar todas as variáveis ao final do programa.
- `getNextLabel` — gera rótulos únicos e sequenciais no formato E_1, E_2, E_3, ... usados em condicionais e laços.

11.2 Descrição das Estruturas de Dados e Algoritmos Utilizados

São utilizados dois mapas (HashMap): `variavelEnderecoMap`, associando cada variável ao seu endereço na pilha, e `variavelTipoMap`, associando cada variável ao seu tipo (usado por `freeMemory` para calcular o tamanho a desalocar). Um contador incremental (`currentTargetCounter`) gera rótulos únicos para desvios de fluxo.

11.3 Alocação de Memória para a Máquina-Objeto

Os tamanhos considerados são os mesmos apresentados na Seção 9.4: `boolean` = 1, `integer` = 2, `real` = 2 unidades de memória.

11.4 Exemplos de Tradução com Trechos Representativos

Comando de Atribuição

Programa-fonte:

```
x := 10 + 5;
```

Código gerado (a alocação de “x” já ocorreu antes, na declaração):


```
LOADL 10
LOADL 5
CALL ADD
STORE 0[SB]
```

Estrutura Condicional

Programa-fonte:

```
if x > 5 then
  y := x
else
  y := 0;
```

Código gerado (rótulos sequenciais reais, testados neste compilador):

```
E_1:
  LOAD 0[SB]
  LOADL 5
  CALL GT
  JUMPIF (0) E_2
  LOAD 0[SB]
  STORE 2[SB]
  JUMP E_3
E_2:
  LOADL 0
  STORE 2[SB]
E_3:
```

12 Manual de Instalação

Para instalar o projeto, obtenha o código-fonte a partir do repositório GitHub (clone ou download do `.zip`) e extraia-o para o diretório desejado. Não há dependências externas além do JDK (*Java Development Kit*).

13 Manual de Compilação

Após obter o projeto, siga os passos abaixo a partir da raiz do repositório:

1. Compilar via terminal:

```
javac -encoding UTF-8 -d bin src/compiler/*.java \
  src/compiler/analise_lexica/*.java \
  src/compiler/analise_sintatica/*.java \
  src/compiler/analise_de_contexto/*.java \
  src/compiler/geracao_de_codigo/*.java \
  src/compiler/visitor/*.java
```

Atenção

O comando de compilação precisa do parâmetro `-encoding UTF-8`, pois as classes contêm acentuação em português nas mensagens de erro; sem esse parâmetro, a compilação falha em ambientes cujo padrão não é UTF-8.

2. Alternativa via VS Code: instale o “Extension Pack for Java”, abra a pasta do projeto, acesse `Compiler.java` e utilize o botão “Run Java”.

3. Alternativa via NetBeans: o repositório já inclui `build.xml`, `manifest.mf` e a pasta `nbproject/`, permitindo abrir e compilar o projeto diretamente pelo NetBeans (*Open Project* → *Build Project*).

14 Manual de Utilização

Requisitos: JDK 8 ou superior instalado (testado também com JDK 17); qualquer sistema operacional com suporte a Java.

Passos para executar:

1. Coloque o arquivo-fonte a ser compilado em um diretório acessível ao projeto;
2. Opcionalmente, passe o caminho do arquivo como argumento de linha de comando (ver abaixo) ou ajuste a variável `Compiler.fileName`;
3. Ajuste a variável `Compiler.steps` conforme a etapa que deseja executar:

Valor de steps	Etapas executadas
1	Análise léxica
2	+ Análise sintática
3	+ Impressão da AST
4	+ Análise de contexto
5	+ Geração de código (padrão)

4. Execute com:

```
java -cp bin compiler.Compiler [caminho-do-arquivo-fonte]
```

Se a compilação for bem-sucedida, o arquivo `Object-code.txt` é gerado na raiz do projeto contendo o código-objeto TAM. Mensagens de erro léxico, sintático ou semântico são exibidas no console com a respectiva linha e coluna, interrompendo a execução no primeiro erro encontrado.

Exemplo Prático de Execução: Abaixo é apresentado um exemplo real de compilação e execução utilizando um arquivo de teste (`test/code0.txt`) no terminal do powershell:

Este código de teste está presente nos arquivos, para maior facilidade de teste, é recomendado apenas alterar o conteúdo dentro dele.

```
PS C:\Users\helli> cd 'C:\Users\helli\OneDrive\Desktop\Matérias\Compiladores\Compilador'
```

```
PS C:\Users\helli\OneDrive\Desktop\Matérias\Compiladores\Compilador> javac -
encoding UTF-8 -d bin src/compiler/*.java src/compiler/analise_lexica/*.
java src/compiler/analise_sintatica/*.java src/compiler/analise_de_contexto
/*.java src/compiler/geracao_de_codigo/*.java src/compiler/visitor/*.java
```

```
PS C:\Users\helli\OneDrive\Desktop\Matérias\Compiladores\Compilador> java -cp
bin compiler.Compiler test/code0.txt
```

---> Impressão da árvore

Programa:

ID: teste5

| | Corpo:

| | Declaracoes:

| | | | | Declarações:

| | | | | Declaração 1:

| | | | | Declaracao:

| | | | | Declaracao de Variavel:

| | | | | IDs:

| | | | | x

| | | | | Tipo:

| | | | | integer

| | | | | Declaração 2:

| | | | | Declaracao:

| | | | | Declaracao de Variavel:

| | | | | IDs:

| | | | | y

| | | | | Tipo:

| | | | | integer

| | Comando composto:

| | | x

| | | Expressão Simples 1:

| | | Termo:

| | | 0

| | | y

| | | Expressão Simples 1:

| | | Termo:

| | | 3

---> Geração de código:

PUSH 2

PUSH 2

LOADL 0

STORE 0[SB]

LOADL 3

STORE 2[SB]

E_1:

LOAD 0[SB]

```

LOAD 2[SB]
CALL LT
JUMPIF (0) E_2

LOAD 0[SB]
LOADL 1
CALL ADD
STORE 0[SB]
JUMP E_1
E_2:

POP 2
POP 2
HALT

```

15 Conclusões Finais

a) Avaliação do projeto, experiência adquirida e contribuição para o aprendizado:

O desenvolvimento deste projeto proporcionou uma compreensão mais aprofundada sobre o funcionamento interno de um compilador. Durante a implementação de cada etapa, foi possível compreender de forma prática como a análise léxica é responsável pela identificação dos tokens da linguagem, como a análise sintática verifica se o programa está de acordo com a gramática definida e como a análise semântica garante a consistência das informações, como declarações de variáveis, tipos e escopos.

b) Principais dificuldades encontradas e encaminhamentos:

A principal dificuldade encontrada durante o desenvolvimento foi garantir que todas as etapas do compilador permanecessem consistentes após a realização das extensões na linguagem. Sempre que uma nova funcionalidade era adicionada à gramática, também era necessário atualizar a análise semântica e a geração de código, exigindo uma verificação cuidadosa para evitar inconsistências entre as diferentes fases do projeto.

Também foram enfrentados problemas relacionados à adaptação da gramática utilizada como base para a versão atual do projeto, além de ajustes na implementação do controle de escopo e da tabela de símbolos. Essas dificuldades foram resolvidas por meio da revisão da gramática, da realização de testes com diferentes programas de entrada e da correção gradual das regras responsáveis pela validação semântica.

Durante o desenvolvimento também foram necessários alguns ajustes de configuração do ambiente, como questões relacionadas ao *encoding* dos arquivos e ao processo de compilação. Após identificar as causas dos problemas, foram realizadas as configurações adequadas do ambiente e novos testes de validação, permitindo que o compilador funcionasse corretamente em todas as etapas previstas.

c) Críticas e sugestões:

Como sugestão de melhoria, considero que a entrega do projeto poderia ser realizada por meio de um repositório online (como GitHub) ou por envio de um arquivo compactado via e-mail ou ambiente virtual da disciplina. Essa forma de entrega facilitaria o envio e a organização dos arquivos, reduziria problemas relacionados à compatibilidade de mídias físicas e permitiria ao professor acessar facilmente o código-fonte e a documentação.

Além disso, a substituição da entrega em CD, DVD ou pendrive e da documentação impressa por versões digitais contribuiria para a redução do consumo de papel e de mídias físicas, tornando o processo mais prático, sustentável e alinhado às práticas atualmente adotadas na maioria dos projetos de desenvolvimento de software.